Informed Search Methods

Heuristic Functions and Advanced Search Algorithms

Machine Intelligence - Lecture 3
Daniel Fišer, Fall 2025

Contents

1 Introduction to Informed Search **2**
1.1 Motivation: Beyond Blind Search . . . 2
1.2 Key Concept: Domain-Specific Knowledge . . . 2

2 Heuristic Functions **2**
2.1 Formal Definition . . . 2
2.2 Properties of Heuristic Functions . . . 3
2.2.1 Admissibility . . . 3
2.2.2 Consistency (Monotonicity) . . . 3
2.3 Examples of Heuristics . . . 4
2.3.1 Example 1: Route Planning (Copenhagen to Prague) . . . 4
2.3.2 Example 2: The 8-Puzzle . . . 4
2.3.3 Example 3: Grid Navigation . . . 5
2.4 Obtaining Heuristics: Relaxation . . . 5

3 Systematic Informed Search **5**
3.1 Greedy Best-First Search (GBFS) . . . 5
3.1.1 Properties of GBFS . . . 6
3.1.2 GBFS Example: Route Finding . . . 7
3.2 A* Search Algorithm . . . 7
3.2.1 Properties of A* . . . 8
3.2.2 A* Example: Route Finding . . . 9
3.3 Comparing GBFS and A* . . . 9

4 Local Search Algorithms **10**
4.1 Motivation: When Systematic Search Fails . . . 10
4.2 Hill Climbing . . . 10
4.2.1 Properties of Hill Climbing . . . 10
4.2.2 Hill Climbing Variants . . . 11
4.3 Hill Climbing Example: 8-Queens Problem . . . 11
4.4 Other Local Search Methods . . . 12
4.4.1 Simulated Annealing . . . 12
4.4.2 Local Beam Search . . . 12
4.4.3 Genetic Algorithms . . . 12

5 Practical Considerations and Comparisons **12**
5.1 Choosing the Right Algorithm . . . 12
5.2 Heuristic Quality . . . 12
5.3 Memory-Bounded A* Variants . . . 13

6 Conclusion and Summary **13**
6.1 Key Takeaways 13
6.2 Algorithm Selection Guide 13
6.3 Further Topics 14

1 Introduction to Informed Search

1.1 Motivation: Beyond Blind Search

In previous lectures, we explored **blind search** (also called **uninformed search**) algorithms such as Depth-First Search (DFS), Breadth-First Search (BFS), and Uniform Cost Search (UCS). These algorithms systematically explore the state space without any knowledge about where the goal might be located.

Limitations of Blind Search

Blind search algorithms:

- Treat all directions of search equally
- May explore large portions of the state space unnecessarily
- Can be inefficient for large or complex problems
- Do not use domain-specific knowledge

Informed search (also called **heuristic search**) improves upon this by using additional knowledge about the problem to guide the search toward the goal more efficiently.

1.2 Key Concept: Domain-Specific Knowledge

Informed search algorithms use a **heuristic function** that estimates how close a state is to the goal. This allows the algorithm to prioritize exploring states that appear more promising.

Example: Route Finding

When finding a route from Copenhagen to Prague:

- **Blind search**: Might explore routes going north toward Norway before considering southern routes
- **Informed search**: Uses straight-line distance to Prague to prioritize exploring cities that are geographically closer to the destination

2 Heuristic Functions

2.1 Formal Definition

Heuristic Function

A **heuristic function** is a mapping:

$$h : \mathcal{S} \to \mathbb{N}_0 \cup \{\infty\}$$

where $\mathcal{S}$ is the state space, and $h(s)$ estimates the cost from state s to the nearest goal state.
Notation:

- $h(s)$: Heuristic estimate for state s
- $h^*(s)$: **Perfect heuristic** (optimal heuristic) - the actual cost of the cheapest path from s to a goal

The perfect heuristic $h^*(s)$ represents the true minimum cost to reach a goal from state s. In practice, we don't know $h^*(s)$ (if we did, we'd already have solved the problem!), so we use approximations.

2.2 Properties of Heuristic Functions

2.2.1 Admissibility

Admissibility

A heuristic function h is **admissible** if:

$$h(s) \leq h^*(s) \quad \text{for all states } s \in \mathcal{S}$$

In other words, an admissible heuristic **never overestimates** the cost to reach a goal.

Why Admissibility Matters

Admissible heuristics are crucial for the optimality of A* search. If a heuristic overestimates the cost to the goal, A* may miss the optimal solution.

Examples of admissible heuristics:

- **Straight-line distance** for route planning (assuming we can't travel faster than a straight line)
- **Manhattan distance** for grid-based movement with only horizontal/vertical moves
- **Number of misplaced tiles** in the 8-puzzle

2.2.2 Consistency (Monotonicity)

Consistency

A heuristic function h is **consistent** (or **monotonic**) if for every state s and every successor s' generated by action a:

$$h(s) - h(s') \leq \text{cost}(a)$$

where $\text{cost}(a)$ is the cost of action a that leads from s to s'.
Equivalently: $h(s) \leq \text{cost}(a) + h(s')$

Consistency Intuition

Consistency is similar to the triangle inequality in geometry. It ensures that the estimated cost doesn't decrease by more than the actual cost of taking an action. This prevents "jumps" in the heuristic values.

Relationship between properties:

Property	Implication
Consistent	$\Rightarrow$ Admissible
Admissible	$\not\Rightarrow$ Consistent

Every consistent heuristic is admissible, but not every admissible heuristic is consistent.

2.3 Examples of Heuristics

2.3.1 Example 1: Route Planning (Copenhagen to Prague)

Straight-Line Distance Heuristic

Problem: Find the shortest driving route from Copenhagen to Prague.
Heuristic: $h(s)$ = straight-line distance from city s to Prague
Why it's admissible: The straight-line distance is always less than or equal to the actual driving distance (you can't drive in a perfectly straight line).
Example values:

- $h(\text{Copenhagen}) = 680$ km
- $h(\text{Berlin}) = 280$ km
- $h(\text{Dresden}) = 120$ km
- $h(\text{Prague}) = 0$ km

2.3.2 Example 2: The 8-Puzzle

The 8-puzzle consists of a 3×3 grid with 8 numbered tiles and one empty space. The goal is to rearrange the tiles from a start configuration to a goal configuration.

8-Puzzle Heuristics

Heuristic 1: Number of Misplaced Tiles

$$h_1(s) = \text{count of tiles not in their goal position}$$

Heuristic 2: Manhattan Distance

$$h_2(s) = \sum_{\text{tiles}} |\Delta x| + |\Delta y|$$

where Δx and Δy are the horizontal and vertical distances of each tile from its goal position.
Comparison:

- Both heuristics are admissible
- h_2 (Manhattan) is generally more informed than h_1 (misplaced tiles)
- $h_2 \geq h_1$ for all states, making h_2 a better heuristic

2.3.3 Example 3: Grid Navigation

Grid Movement Heuristics

For navigating a 2D grid from position (x_1, y_1) to goal (x_g, y_g):
If only 4-directional movement allowed (N, S, E, W):

$$h(s) = |x_1 - x_g| + |y_1 - y_g| \quad \text{(Manhattan distance)}$$

If 8-directional movement allowed (including diagonals):

$$h(s) = \max(|x_1 - x_g|, |y_1 - y_g|) \quad \text{(Chebyshev distance)}$$

If any-angle movement allowed:

$$h(s) = \sqrt{(x_1 - x_g)^2 + (y_1 - y_g)^2} \quad \text{(Euclidean distance)}$$

2.4 Obtaining Heuristics: Relaxation

One common method for deriving admissible heuristics is through **problem relaxation**.

Relaxation

A **relaxed problem** is a version of the original problem with fewer constraints or restrictions. The cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem.

Relaxation: 8-Puzzle

Original problem: Move tiles into adjacent empty space
Relaxation 1: A tile can move to any adjacent square (even if occupied)

- Optimal solution cost = Manhattan distance
- Yields h_2 heuristic

Relaxation 2: A tile can move to any square

- Optimal solution cost = number of misplaced tiles
- Yields h_1 heuristic

3 Systematic Informed Search

3.1 Greedy Best-First Search (GBFS)

Greedy Best-First Search is the simplest informed search algorithm. It expands the node that appears to be closest to the goal, according to the heuristic function.

Greedy Best-First Search Algorithm

Algorithm 1 Greedy Best-First Search

```
function GBFS(Θ, h)                                   ▷ Θ is the problem, h is the heuristic
    n_0 ← InitNode(Θ)                                 ▷ Create initial node
    open ← priority queue ordered by h(n.State) ascending
    open.Insert(n_0)
    closed ← ∅
    while open ≠ ∅ do
        n ← open.Pop()                                ▷ Node with smallest h value
        if GoalTest(n.State) then
            return ExtractPath(n)
        end if
        closed ← closed ∪ {n}
        for each a ∈ Actions(n.State) do
            n′ ← SuccNode(n, a)
            if n′.State ∉ closed and n′ ∉ open then
                open.Insert(n′)
            end if
        end for
    end while
    return Failure
end function
```

3.1.1 Properties of GBFS

Property	Value	Notes
Complete?	No	Can get stuck in loops
Optimal?	No	Greedily follows heuristic
Time Complexity	$O(b^m)$	m = max depth, b = branching factor
Space Complexity	$O(b^m)$	Keeps all nodes in memory

GBFS Limitations

Not complete: Can get stuck in infinite loops if there are cycles
Not optimal: May find a goal quickly, but not necessarily the cheapest path

3.1.2 GBFS Example: Route Finding

GBFS: Arad to Bucharest

Consider finding a route from Arad to Bucharest in Romania using straight-line distance as the heuristic.

Step 0: Initialize

Open (ordered by h)	Arad (366)
Closed	$\emptyset$

Step 1: Expand Arad

- Pop: Arad ($h = 366$)
- Not goal
- Successors: Sibiu ($h = 253$), Timisoara ($h = 329$), Zerind ($h = 374$)

Open	Sibiu (253), Timisoara (329), Zerind (374)
Closed	Arad

Step 2: Expand Sibiu

- Pop: Sibiu ($h = 253$)
- Successors: Fagaras ($h = 176$), Rimnicu Vilcea ($h = 193$)

Open	Fagaras (176), RV (193), Timisoara (329), Zerind (374)
Closed	Arad, Sibiu

Step 3: Expand Fagaras

- Pop: Fagaras ($h = 176$)
- Successors: Bucharest ($h = 0$) $\leftarrow$ **GOAL!**

Solution found: Arad $\rightarrow$ Sibiu $\rightarrow$ Fagaras $\rightarrow$ Bucharest

Note: This path has cost 450, but there exists a cheaper path (Arad $\rightarrow$ Sibiu $\rightarrow$ Rimnicu Vilcea $\rightarrow$ Pitesti $\rightarrow$ Bucharest with cost 418). GBFS found a solution, but not the optimal one.

3.2 A* Search Algorithm

A* (pronounced "A-star") combines the benefits of uniform cost search and greedy best-first search. It considers both the cost to reach a node and the estimated cost to the goal.

A* Evaluation Function

A* uses the evaluation function:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ = cost of the path from the start to node n
- $h(n)$ = heuristic estimate from n to the goal
- $f(n)$ = estimated total cost of the cheapest solution through n

A* Search Algorithm

Algorithm 2 A* Search

```
function ASTAR(Θ, h)                                  ▷ Θ is the problem, h is the heuristic
    n_0 ← INITNODE(Θ)
    g(n_0) ← 0
    f(n_0) ← h(n_0.State)
    open ← priority queue ordered by f(n) ascending
    open.INSERT(n_0)
    closed ← ∅
    while open ≠ ∅ do
        n ← open.POP()                                ▷ Node with smallest f value
        if GOALTEST(n.State) then
            return EXTRACTPATH(n)
        end if
        closed ← closed ∪ {n}
        for each a ∈ ACTIONS(n.State) do
            n′ ← SUCCNODE(n, a)
            g(n′) ← g(n) + cost(a)
            f(n′) ← g(n′) + h(n′.State)
            if n′.State ∈ closed then
                if g(n′) < g(old) then                ▷ Found cheaper path
                    Remove old node from closed
                    open.INSERT(n′)
                end if
            else if n′ ∈ open then
                if g(n′) < g(old) then
                    Update n′ in open with new lower g and f values
                end if
            else
                open.INSERT(n′)
            end if
        end for
    end while
    return FAILURE
end function
```

3.2.1 Properties of A*

Property	Value	Condition
Complete?	Yes	If b is finite and costs $> \epsilon > 0$
Optimal?	Yes	If h is admissible
Time Complexity	$O(b^d)$	Exponential in solution depth
Space Complexity	$O(b^d)$	Keeps all nodes in memory

A* Optimality Guarantee

Theorem: If the heuristic h is admissible, then A* is guaranteed to find an optimal solution.
Proof intuition: Because h never overestimates, A* will never overlook a cheaper path. It expands nodes in order of increasing f values, ensuring that when a goal is found, no cheaper path exists.

3.2.2 A* Example: Route Finding

A*: Arad to Bucharest

Using the same problem as GBFS, but now considering both path cost g and heuristic h.
Step 0: Initialize

Node	g	h	$f = g + h$
Arad	0	366	366

Step 1: Expand Arad Successors:

- Sibiu: $g = 140$, $h = 253$, $f = 393$
- Timisoara: $g = 118$, $h = 329$, $f = 447$
- Zerind: $g = 75$, $h = 374$, $f = 449$

Open list (ordered by f): Sibiu (393), Timisoara (447), Zerind (449)
Step 2: Expand Sibiu Successors:

- Fagaras: $g = 140 + 99 = 239$, $h = 176$, $f = 415$
- Rimnicu Vilcea: $g = 140 + 80 = 220$, $h = 193$, $f = 413$

Open list: RV (413), Fagaras (415), Timisoara (447), Zerind (449)
Step 3: Expand Rimnicu Vilcea Successors include:

- Pitesti: $g = 220 + 97 = 317$, $h = 100$, $f = 417$

Open list: Fagaras (415), Pitesti (417), Timisoara (447), Zerind (449)
Step 4: Expand Fagaras Successors:

- Bucharest: $g = 239 + 211 = 450$, $h = 0$, $f = 450$

Open list: Pitesti (417), Bucharest (450), Timisoara (447), Zerind (449)
Step 5: Expand Pitesti Successors:

- Bucharest: $g = 317 + 101 = 418$, $h = 0$, $f = 418$

Update: Better path to Bucharest found! Update $f = 418$
Open list: Bucharest (418), Timisoara (447), Zerind (449), Bucharest (450, old)
Step 6: Pop Bucharest with $f = 418$ **Goal reached!**
Optimal solution: Arad $\rightarrow$ Sibiu $\rightarrow$ Rimnicu Vilcea $\rightarrow$ Pitesti $\rightarrow$ Bucharest
Total cost: 418

3.3 Comparing GBFS and A*

Property	**GBFS**	**A***
Evaluation function	$f(n) = h(n)$	$f(n) = g(n) + h(n)$
Complete?	No	Yes (with conditions)
Optimal?	No	Yes (if h admissible)
Speed	Often faster	May expand more nodes
Memory	High	High

When to Use Which Algorithm

Use GBFS when:

- You need a quick solution (not necessarily optimal)

- The heuristic is very reliable
- Memory is extremely limited

Use A* when:

- Optimality is required
- You have an admissible heuristic
- You have sufficient memory

4 Local Search Algorithms

4.1 Motivation: When Systematic Search Fails

For many problems, systematic search algorithms like A* are impractical due to:

- **Huge state spaces**: Billions or trillions of states
- **Memory limitations**: Cannot store all explored nodes
- **Goal state irrelevant**: Only care about optimization, not path

Local search algorithms operate by maintaining a current state and iteratively moving to neighboring states to improve an objective function.

4.2 Hill Climbing

Hill Climbing is the simplest local search algorithm. It continuously moves to the neighbor with the best value until no improvement can be made.

Hill Climbing Algorithm

Algorithm 3 Hill Climbing

```
function HillClimbing(Θ, h)                          ▷ Minimize h to find goal
    current ← InitialState(Θ)
    while true do
        neighbors ← GenerateNeighbors(current)
        next ← arg min_{s∈neighbors} h(s)
        if h(next) ≥ h(current) then                 ▷ No improvement
            return current
        end if
        current ← next
    end while
end function
```

4.2.1 Properties of Hill Climbing

Property	Value	Notes
Complete?	No	Can get stuck in local optima
Optimal?	No	Finds local optimum, not global
Time Complexity	Varies	Depends on landscape
Space Complexity	$O(1)$	Only stores current state

Hill Climbing Problems

Hill Climbing can get stuck in:

- **Local maxima/minima**: A peak that is not the highest
- **Plateaus**: Flat areas where all neighbors have the same value
- **Ridges**: Areas where progress requires moving through worse states

4.2.2 Hill Climbing Variants

1. Random-Restart Hill Climbing:

- Run hill climbing multiple times from random initial states
- Return the best solution found
- Complete if there are finite local maxima

2. Stochastic Hill Climbing:

- Choose randomly among uphill moves
- Probability weighted by steepness of improvement

3. First-Choice Hill Climbing:

- Generate successors randomly until one is better
- Good when there are many neighbors

4.3 Hill Climbing Example: 8-Queens Problem

8-Queens with Hill Climbing

Problem: Place 8 queens on a chessboard so that no two queens attack each other.
State representation: One queen per column, state specifies row positions
Heuristic: $h(s)$ = number of pairs of queens that attack each other
Goal: $h(s) = 0$
Neighbors: Move one queen to a different row in its column
Initial state example:

```
State: [2, 4, 7, 4, 8, 5, 5, 2]
h = 17 (17 pairs of attacking queens)
```

Step 1: Try moving each queen

- Move queen in column 1 to different rows: best improvement moves h to 14
- Move queen in column 5 to row 6: h becomes 12

Current state after improvement:

```
State: [2, 4, 7, 4, 6, 5, 5, 2]
h = 12
```

Continue until $h = 0$ or stuck at local minimum.
Note: Hill climbing solves about 14% of 8-queens instances from random starts. With random restarts, it can solve nearly all instances quickly.

4.4 Other Local Search Methods

4.4.1 Simulated Annealing

Idea: Allow some "bad" moves to escape local optima, with probability decreasing over time.

- If move improves state: always accept
- If move worsens state: accept with probability $e^{-\Delta E/T}$
- T (temperature) decreases over time according to a schedule

4.4.2 Local Beam Search

Idea: Keep track of k states instead of just one.

- Generate all successors of all k states
- Keep the best k successors
- Like k parallel hill climbs, but information shared

4.4.3 Genetic Algorithms

Idea: Inspired by biological evolution.

- Population of states (individuals)
- Select fittest individuals
- Combine (crossover) and mutate to create new generation
- Repeat until solution found or time limit

5 Practical Considerations and Comparisons

5.1 Choosing the Right Algorithm

Algorithm	Optimal?	Complete?	Space	When to Use
BFS	Yes*	Yes	High	Small space, uniform cost
UCS	Yes	Yes	High	Non-uniform costs
GBFS	No	No	High	Quick solution needed
A*	Yes**	Yes**	High	Optimal solution required
Hill Climb	No	No	Low	Huge space, local ok

*BFS optimal only if all costs equal
**A* optimal/complete if heuristic is admissible

5.2 Heuristic Quality

Effective Branching Factor

The **effective branching factor** b^* measures the quality of a heuristic. If A* expands N nodes to depth d, then b^* satisfies:

$$N = 1 + b^* + (b^*)^2 + \cdots + (b^*)^d$$

Lower b^* indicates a better heuristic.

Dominance: If $h_2(s) \geq h_1(s)$ for all states s, and both are admissible, then h_2 **dominates** h_1 and is better for search.

5.3 Memory-Bounded A* Variants

IDA* (Iterative Deepening A*):

- Uses f-cost limit instead of depth limit
- Space complexity: $O(d)$
- Time complexity: Similar to A* in many domains

SMA* (Simplified Memory-Bounded A*):

- Uses all available memory
- Drops worst leaf node when memory full
- Regenerates nodes when needed

6 Conclusion and Summary

6.1 Key Takeaways

- **Informed search** uses domain knowledge via heuristic functions to guide search more efficiently than blind search
- **Admissible heuristics** never overestimate the cost to goal, ensuring A* finds optimal solutions
- **Consistent heuristics** satisfy a triangle inequality property and are always admissible
- **Greedy Best-First Search** is fast but not optimal; it expands nodes with smallest h value
- **A* Search** is optimal and complete (with admissible h); it expands nodes with smallest $f = g+h$
- **Local search** trades completeness and optimality for low memory usage; suitable for huge state spaces
- **Hill Climbing** is the simplest local search but can get stuck in local optima
- **Problem relaxation** is a key method for deriving admissible heuristics

6.2 Algorithm Selection Guide

Scenario	**Recommended Algorithm**
Need optimal solution, have admissible heuristic, sufficient memory	A*
Need quick solution, optimality not critical	GBFS
Huge state space, limited memory, local optimum acceptable	Hill Climbing or Simulated Annealing
Optimization problem, no clear goal state	Local Search methods
Want optimal, limited memory	IDA* or SMA*
No domain knowledge available	BFS or UCS (uninformed)

6.3 Further Topics

Topics not covered in this lecture but worth exploring:

- Bidirectional A* search
- Pattern databases for puzzle heuristics
- Parallel and distributed search
- Anytime algorithms (improve solution over time)
- Constraint satisfaction problems (CSP)
- Adversarial search (game playing)

End of Lecture 3 Notes